把七项要求放进同一个可验证的调度器

本轮重新核对公开题面、检索一手论文和工程文档，实现七组策略消融，再完成两次冻结后的新样例验证与大规模计时。最终保留保守自适应主版，并提供动作等价的扫描优化候选；额外选盘搜索与回读债务作为研究候选交付。

本轮交付	已完成的证据
跨算子、next_op、自适应	主版已有全局选盘、软配额、在线规则切换、受限联合预取及反馈补偿
新增原创方向	阶段保护、强制落盘集合证书、可见成本预算搜索、跨算子回读债务
算法与CPU分别消融	876例开发集、32例定向增长反例、420例新验证、388例再次新验证
合法性与行为核对	12,728次完整逐步回放；扫描合并版与原版在1,716条完整输入上动作一致
真实计时	公开引擎原计时；策略验证各3轮、最终6个规模输入各5轮

当前结论

小候选精细搜索降低部分I/O，却增加在线CPU；420例完整Score几何比相对前版仅改善约0.0043%，不足以宣布稳定领先。扫描合并版在500万混合输入中决策时间降低3.60%，但另有负载退化，仍不强制替换主版。

这里的“正确”分成三层：动作合法、局部性质可证明、本地完整Score实测。任何一层都不能推出官方隐藏测试满分或全输入最优。

逐项覆盖，
每一点都有实现或对照

要求	具体落实	结果与状态
1 跨算子落盘	全局victim集合；算子软配额；新增按真实读回成本累计的债务	主版保留；债务模块单独实验，未默认开启
2 决策时间tradeoff	统一用I/O+罚时+决策时间评估；仅在收益上界值得搜索时求精	精细搜索实测有正有负，保留完整数据
3 回调代码优化	预留动作容量、精确单/双候选快路径；新增融合驻留统计与候选收集	1,716例动作等价；6个规模输入5轮计时
4 next_op提前决策	当前miss联合下一PROBE；周期性准备下一INSERT；无额外落盘预取与强制集合预取	只见一条next_op；LOAD顺序合法；新增BUILD保护
5 学术/业界搜索	DuckDB、Velox、LHD、S3-FIFO、LeCaR及内存仲裁压力测试	一手来源与适配边界见第3页和来源JSON
6 动态感知与自适应	阶段、累计miss增量、短间隔读回、衰减热度；事件触发重评与滞回	首次miss初始化；后续低成本观察和按条件重评
7 自主创新	预取反馈补偿、阶段非放大门槛、强制集合证书、计算预算与回读债务	区分已证明的局部性质、工程改进与未证实的设想

主版并非为BUILD、PROBE各写一套永久固定规则。阶段只影响默认偏好与代价权重；每次必要决策都会读取当前驻留、大小和下一操作。较重的规则选择与配额重算使用事件门槛。

文献能借鉴什么，本题还需要补什么

R1 Saving Private Hash Join

统一内存视角、并存算子的运行时容量管理；在明确时点做较重的调整。 题目固定哈希分区与整分区动作，不能复现论文的压缩、缓冲页设计或重分区。

https://www.vldb.org/pvldb/vol18/p2748-kuiper.pdf

R2 Velox Memory Management

随使用量变化的容量仲裁，可回收其他池的内存。借鉴为全局候选与算子软配额。 本题没有可修改容量、线程或独立查询池的接口；软配额不改变题目的全局M。

https://facebookincubator.github.io/velox/develop/memory.html

R3 Velox Spilling

区分写出与恢复，按分区选择、协调多个Build实例；借鉴为读写代价分账和跨算子候选。 本题不允许改动文件布局、压缩或执行器；每次SPILL均按题面收费。

https://facebookincubator.github.io/velox/develop/spilling.html

R4 LHD: Improving Cache Hit Rate by Maximizing Hit Density

用单位空间的预期复用价值选victim，并关注统计维护成本。 这里没有逐次hit回调，热度是偏采样观测；未复现LHD条件概率模型。

https://www.usenix.org/conference/nsdi18/presentation/beckmann

R5 FIFO queues are all you need for cache eviction / S3-FIFO

低维护开销、ghost辨认再次访问、惰性维护。 不能照搬依赖所有hit更新的缓存策略；不引用其缓存命中收益作为本题结果。

https://s3fifo.com/

R6 Driving Cache Replacement with ML-based LeCaR

利用驱逐后的反馈在替换规则之间调整。 当前实现为带滞回的经验控制器，不声称复现强化学习或有无遗憾界。

https://www.usenix.org/conference/hotstorage18/presentation/vietri

R7 Velox MemoryArbitration Fuzzer

内存压力下的多算子随机和压力验证。 只借鉴测试思想；本题M保持固定，没有添加并发线程。搜索索引可读，直接打开曾失败。

https://facebookincubator.github.io/velox/develop/testing/memory-arbitration-fuzzer.html

R1在固定同步时点分配内存，并用有限次数梯度更新控制优化开销；本题接口不同，因此借鉴“少量关键时点做重优化”的思想，没有声称复现其代价模型。R7的搜索索引可读，直接打开失败，测试思想不依赖该页的细节。

跨算子：
全局候选、软配额与回读债务

基础模型

把所有算子的驻留非空分区放入一个候选集合，排除当前INSERT目标及显式保护目标。缺口D=本次新增驻留字节-(M-memory_used)。D≤0直接返回；D>0时选出释放字节合计至少D的完整分区。

BUILD默认使用最大分区优先：按-size/配额因子排序，通常较少动作便可释放足够空间。PROBE在观测时间、观测热度/字节、预期恢复代价之间在线选择。最大分区是全局可驱逐集合中的最大项，并不限于请求算子。

算子软配额

容量上界为该算子当前数据总量，权重为阶段系数×(1+衰减观测热度)。采用有上界的加权水位分配；将(预算+平滑量)/(驻留量+平滑量)截断到[0.5,2]后修正选盘代价。它允许借用闲置容量，不把预算当成硬约束。

本轮新增：回读债务

若算子A为自身需求驱逐了算子B的PROBE分区，而且victim与需求在容量上本来可以共存，那么B短时间内被实际LOAD时，累计 size/W+size/R+32 的观测恢复成本。债务按Probe时钟衰减，再把B的软配额权重最多放大3倍。

这比只计“跨算子驱逐次数”更能区分大分区和慢读盘。但容量可共存并不代表本次驱逐一定可避免；无法观察的未来也使它不是严格的因果后悔值。

新420例 / 相对前版	收益与代价
累计miss	104,075 → 103,979，少96次
完整Score几何比	1.000634，整体略差
最佳与最差例	hot_large_00降低15.98%；empty_partitions_06增加12.08%
当前处理	代码已实现、单独对照；不加入默认主版

动态感知：每次观察，按事件重评

运行位置	做什么	开销控制
每次overflow	读取当前大小/驻留；全局选择；BUILD时按需要重算配额	没有固定地等待下一次周期
首次真实miss	用当前目标与next_op的大小特征初始化规则	无需预先知道数据分布
后续miss	处理ghost读回、观测当前/下一目标；检测阶段与窗口变化	未触发门槛便跳过重评
规则重评触发	阶段变化；128次Probe；32次有效压力miss；或3次短间隔读回	紧急切换最少间隔8次Probe，普通为64次
周期回调	准备已知下一INSERT；观察下一PROBE并尝试受限预取	B由框架决定，没有改大B来隐藏开销

这里学到的是什么

学习的是有限可见信息下的复用证据、算子压力与规则的短间隔读回损失。热度随Probe时钟衰减；ghost记录哪个规则驱逐了谁、多久后读回；规则损失采用指数平滑，并以滞回避免频繁切换。本轮没有训练神经网络，也没有把未知操作序列交给模型。

PROBE代价规则

C_i=[s_i/W + q_i×(s_i/R+32)]×配额因子，按C_i/s_i排序；q_i=0.15+0.70h_i/(1+h_i)，BUILD再乘0.35，已知下一PROBE目标取1。32是固定启发式尺度，既不是题目隐藏罚时，也不是已校准的真实概率。

自适应的观测边界

普通hit没有回调，无法获取完整LRU/LFU序列；next_op与周期回调仅提供部分样本。可从hit_count+miss_count获得时间增量，但不能把聚合命中数归属到某个分区。报告与实现均保留这个限制。

预取补偿

联合预取避免B的miss后，仍把这次被替代的压力加回控制器计数，防止“预取成功”被误判成“换入压力消失”。只重建同状态下单目标前缀能否免费预取B，不模拟整个隐藏未来。

next_op：提前做事，也约束额外落盘

具体例子

M=64MiB，驻留8个8MiB分区；当前miss目标A、紧邻下一目标B均为8MiB磁盘PROBE分区。若当前在线规则已选择热度/字节，且候选满足保守门槛，先为16MiB选出两个victim，再返回SPILL X、SPILL Y、LOAD B、LOAD A。随后B命中。

若单独处理A本来只释放8MiB，B原本还会miss；联合处理可少一次罚时和一次重复计划。该例在victim及后续行为使两种方案I/O相同的前提下说明收益，不能推出每次联合都更便宜。

保守联合条件	理由
A+B≤M/2；A/B相差不超过2倍	给其余工作集留容量，避免小请求牵动大装载
每个victim≤max(A,B)+容差	保护大驻留分区
总写出≤A+B+容差	限制容量贪心的超额写出
新增：额外前缀不得包含BUILD victim	避免联合提前加载额外触发BUILD不可逆落盘
最后动作LOAD A	保持当前miss接口要求；B可来自其他算子

新增阶段保护的局部性质

联合仅在热度/字节规则下启用，victim顺序与同状态单目标的贪心前缀相同。先累计只够A所需的前缀；若后续额外victim含BUILD便拒绝联合。因此，获准的联合不因扩大到A+B而多驱逐一个BUILD分区。它不保证未来I/O或完整Score不增。

实际覆盖情况

本轮876开发例、420新例和32定向增长例中的保护拒绝计数均为0。这个门槛在已测输入里没有提供可测收益，不能把它作为成绩提高的原因。定向样例仍保留，避免用“写了保护代码”代替触发证据。

其他提前操作：无额外victim时免费预取；周期性预取仅在强制victim集合够用时执行；下一INSERT确将溢出时可提前准备空间。没有看见B之后的C。

计算也收费：让搜索有预算、有退出条件

真实目标

Score=T_IO + p×Miss + T_decision。是否多花δ微秒求更精细的方案，应比较预期I/O/罚时节省与δ；候选模型变小不等于真实Score变小。公开引擎的状态快照耗时也处于计时范围，本轮没有移动或删减它。

新增小候选搜索

仅对PROBE代价规则、候选数2到10尝试枚举，至多1,024个子集；目标是以最小ΣC_i覆盖D。原方案作为始终可用的incumbent。若完整枚举结束，知道当前模型的最优子集；若被时间门槛中断，只保留已见最优候选。

搜索入口的收益上界

LB=D×min(C_i/s_i)是覆盖模型的下界，因此 incumbent_cost-LB 是可能改善量的上界。若上界不超过max(0.25微秒,2×搜索耗时EMA)，跳过。它只是入口筛选，不是对未来收益的证明。

只用可见量计算预算

L=total_io_bytes/max(W,R)+miss_count。因每个字节按W或R收费且p≥1，有L≤累计非CPU成本。搜索体累计预算暂取0.001L；预计再花4个EMA仍超预算就不开始。枚举每64个子集读时钟，结束后更新EMA。

该预算的严格边界

预算只约束枚举体的实测累计时间，是软门槛；入口检查、候选收集、时钟调用及最后一个64子集块都会额外收费，系统抢占也可能超时。不能把它宣传成“整个回调最多占Score的0.1%”。

接受条件

只有模型改进超过max(0.25微秒,2×本次实际搜索体时间)才采用新子集。新420例中组合版完成12,595次搜索、接受1,445次，但累计决策时间仍增加4.61%，说明预算与入口判断仍有优化空间。

局部证明与代码优化：让可验证的性质落到实现

必落盘集合证书

当前需要最终驻留的目标大小为S。任何已驻留分区x若size(x)>M-S，就不可能与目标同时驻留，完成LOAD前必须SPILL。若全部这些强制分区的总字节已经覆盖缺口D，直接返回它们，无需排序或选取额外分区。

证明：若某个这样的x留在内存，则S+size(x)>M，与完成动作后的容量约束矛盾。集合已足够时，所有可行方案都包含该集合，加入其他正大小分区只会增加本次写出。因此该集合对本次写出字节最小。该证明不涉及未来访问；输出顺序与统计维护变化也可能影响后续行为。

主版保留的精确快路径

按原比较器线性找出最前一项或两项。若它们恰能覆盖原贪心前缀，继续执行原单分区比较、冗余删除与平局处理，省去全排序。其结果与原启发式一致，不是背包覆盖的全局最优。最坏仍回退O(Q log Q)，Q≤512。

本轮新增：合并两次全表扫描

原实现先遍历Q个分区统计每个算子驻留量，再遍历Q个分区收集victim并计算key。新实现一次遍历同时统计驻留量并收集候选，然后只在n个有效候选上计算key。表读取从约2Q减少到Q+n；n Q时更有机会受益。

等价性条件

驻留量相加顺序、候选顺序、浮点表达式、比较器和平局规则全部保留。配额计算所需输入相同，key相同，后续计划相同，因此输出动作相同。小n枚举模块受时钟影响，未把这个证明扩展到带时钟的组合版。

验证结果

1,716条完整输入、3,432次独立回放，对齐完整动作哈希、读写字节、Hit/Miss、回调数及动作数，全部一致。另对6个规模输入逐动作验证，驻留总量每4,096步复核；两版动作哈希也一致。

876例开发消融：组合收益不能靠猜

开发配置 / 各1轮	Score几何比 相对Naive，越低越好	单组公式均值 越高越好	累计CPU/微秒
前版	0.617817	2.321742	452,565.86
阶段保护	0.617107	2.322019	449,973.71
保护+回读债务	0.617323	2.330998	450,116.20
保护+子集搜索	0.615769	2.324655	474,836.52
保护+强制集合	0.617572	2.323003	443,666.95
保护+债务+搜索	0.615720	2.326722	483,490.99
保护+搜索+强制集合	0.615582	2.326605	470,494.98
全部模块	0.618410	2.328365	464,895.07

据开发集选择保护+搜索+强制集合（特征13），冻结源文件后再生成420例。阶段保护单独没有改变I/O和miss；加入全部模块并没有自动形成最优配置。

开发计时只有一轮，用于筛选，不用于宣称速度优势。全部组合的最差比值曾出现58.36倍：核查为零I/O、零miss的小输入上58.94微秒对1.01微秒的计时异常，不能解释为算法产生了58倍I/O。后续使用多轮原始完整Score。

严格回放的模块活动

模块	开发集活动计数
回读债务更新	66,720
搜索 / 组合版	39,262次开始，4,545次接受
强制集合 / 组合版	55,217次直接计划
阶段保护拒绝	0次；没有实测贡献证据

420例新验证：局部收益与退化一起看

配置 / 各3轮中位数	前版	回读债务	搜索+强制集合
Score几何比/Naive	0.646491	0.646901	0.646463
单组公式均值	1.961333	1.954905	1.967228
累计决策/微秒	242,424.35	242,357.79	253,608.06
累计miss	104,075	103,979	104,736
最差Score/Naive	1.5492	1.5492	1.5492

420例中419例Naive Score>0，另一例不参加比值统计。几何比先对每例使用该配置完整Score的中位数，再相对同轮组Naive取比；单组公式为10×max(0,1-VAL/BASE)。这不是官方50分聚合后的成绩。

搜索组合相对前版：累计I/O时间降低约0.30%，miss增加661次，累计CPU增加4.61%；完整Score几何比0.9999567。112例变差、83例变好，其余为相等或很小差异。整体改善很小，不能确认稳健优势。

组合版相对前版的实际案例	完整Score变化	对应非CPU变化
hot_large_00	-27.35%	I/O少300,153,802.61微秒；miss少15
corner_072	-16.49%	I/O少184,727,836.36微秒；miss多2
build_heavy_02	+21.47%	I/O多40,131,563.21微秒；miss多4
hot_plus_scan_09	+12.47%	I/O多2,338,956.59微秒；miss多25

组合改变的是当前模型最优覆盖，并未知道真实未来；因此“当前代价模型更优”仍能导致未来写读次数更多。这是精细求解的核心边界。

验证后不调整该组合；这420例此后作为已见回归集使用，后续扫描优化另行冻结，再生成388例。

扫描优化：收益来自少读一次分区表

再次冻结后的388例，各3轮

指标	前版	扫描合并版
累计决策 / 微秒	318,846.07	319,739.20
Score几何比/Naive	0.662634	0.663000
累计miss	93,003	93,003
I/O与动作	相同	1,716例完整动作相同

新例累计CPU增加0.28%，Score几何比相对前版增加0.0554%；没有通用领先。386例可算比值，另2例BASE=0。少量亚微秒到数微秒输入的比值明显受计时粒度和系统状态影响，仍保留原数据。

六个规模输入，各5轮中位数

输入	前版CPU/微秒	合并版CPU/微秒	CPU变化
100万次热命中 / B1	286,294.82	286,304.76	+0.00%
512分区压力 / B1	76,256.12	74,149.08	-2.76%
512分区压力 / B1000	61,615.37	53,709.22	-12.83%
500万操作全驻留 / B1	1,450,103.97	1,455,803.02	+0.39%
500万操作混合换入	3,154,205.77	3,040,744.84	-3.60%
500万操作 / 4GiB增长	1,420.60	1,444.71	+1.70%

混合500万输入：决策时间降低3.60%，完整Score由5,104,015,939.83降至5,103,902,478.90，实际Score只下降约0.00222%；该场景I/O与罚时占主导。B1000压力场景CPU降低12.83%、Score降低0.642%。全驻留500万输入则Score增加0.393%。

因此交付可选的等价优化文件，主版保持不变；不把CPU百分比直接当成Score百分比。

验证边界与复现：不改计时，不使用隐藏信息

检查	本轮结果
题面版本	2026-10-26重新获取公开API，HTTP200；保留HTML、纯文本、原JSON与来源记录
规则合法性	只能SPILL驻留分区、LOAD磁盘PROBE分区；每步容量不超M；miss最后LOAD当前目标；排除当前INSERT目标
全步骤回放	12,728次，全部通过；含开发876、新420、定向32与再次新388例
规模核对	6个输入12次独立执行，动作逐项检查，驻留总量每4096步复核，动作哈希一致
源文件与输入	两阶段源码冻结哈希未变；新系统熵与每例SHA256均记录；总计1,722个不同输入文件
公开执行器	核对源码仅增加调度器include和替换实例；快照、回调、周期与Score计算不变
C++约束	单文件、无main、默认构造、C++14标准库；g++13.3，-O2 -Wall -Wextra -pedantic-errors
随机性与时钟	输入先冻结再从系统熵生成；搜索版决策依赖实测时钟，本批重复无I/O或miss分叉，但不保证跨机确定性

怎么复现

现有Ubuntu中运行 experiment.py 接入公开引擎；严格回放使用strict_replay或strict_fused；audit.py只读校验冻结哈希、每例输入哈希、计时结果与动作一致性。README.txt提供具体命令与文件用途。

不要直接重跑validate.py、validate_fused.py再把重生成数据说成原测试；它们会产生新熵并覆盖相应验证目录。保存的manifest与原始jsonl是本轮证据。报告数值由这些文件计算，未剔除失败或变差样例。

官方第20题：内存受限下Hash Join算子Spill-to-Disk动态调度

自主贡献与当前建议

自己的具体贡献	已做到什么	严格边界
预取后的压力反馈补偿	用单目标前缀重建被联合预取替代的miss	同状态局部比较；非完整反事实回放
BUILD阶段非放大门槛	拒绝联合新增的BUILD victim	已测拒绝0次，尚无经验收益
必落盘集合证书	强制集合够用时直接计划；证明本次写出集合最小	后续Score可能改变；不能宣布全局最优
可见成本下界预算搜索	用I/O字节与miss构造可见下界，按搜索耗时EMA限制枚举	预算是搜索体软门槛，模型改善不保证真实收益
跨算子回读债务	把实际恢复成本回馈到victim算子的软配额	已实现、有正负结果；不是因果归因证明
候选收集扫描合并	保持数学运算与动作顺序；大Q压力场景节省CPU	普通样例未稳定更快，作为可选优化

“原创”指本次针对受限接口提出、实现和验证的具体设计，基础工具来自上述文献；没有做全球专利/论文穷尽检索，不声称全球首次。

为什么不能承诺全输入最优

M=64，驻留X/Y各32，A在磁盘且大小32。当前请求A、next_op仍为A；此时驱逐X或Y的可见信息可以完全相同。但再下一步可能访问X，也可能访问Y。固定在线选择总有一种后续让被驱逐者立刻读回。只看一条next_op不能保证两种未来同时最优。

当前交付选择

默认仍为MySpillScheduler_final.cpp（保守联合、动态规则与精确双候选快路径）。MySpillScheduler_scan_fused.cpp是本轮动作等价的可选CPU优化；MySpillScheduler_holistic.cpp为搜索+强制集合的研究候选；pressure.cpp单独保留回读债务实验。每次只选择一个MySpillScheduler类提交。

本轮的有效推进是：七项要求都有实现路径和验证位置，新增局部可证明的选择规则，以及有收益边界的代码优化；没有把更复杂的组合自动标成更优，也没有用文献成绩替代本题实测。